Capítulo 2

Métodos Directos para la resolución de Sistemas de Ecuaciones Lineales

Introducción

Consideremos el siguiente problema clásico:

La edad de una niña y su madre suman 40 años. Hace 10 años, la edad de la madre doblaba a la de su hija. ¿Cuál es la edad de ambas?

Para pensar cómo resolverlo, se puede ver que se tienen dos incógnitas: la edad M de la madre y la edad m de su hija. El problema, a su vez, plantea dos condiciones, que se pueden representar como dos ecuaciones: la primera dice que la suma de las edades es 40 ($M + m = 40$); la segunda dice que la edad de la madre hace 10 años (es decir, a la edad actual le restamos 10) es dos veces la edad de la hija (también considerada hace 10 años). Esta segunda condición la planteamos con la ecuación $M - 10 = 2(m - 10)$.

Luego, nos queda planteado el siguiente sistema de dos ecuaciones con dos incógnitas:

$$\begin{aligned} M + m &= 40 \\ (M - 10) &= 2(m - 10) \end{aligned}$$

Al plantear diferentes modelos para la resolución de problemas de la ingeniería, se necesitará resolver sistemas de ecuaciones algebraicas lineales (SEAL). Estos sistemas podemos escribirlos como sigue:

$$\begin{array}{rrcl} E_1: & a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \ldots + a_{1n}x_n & = & b_1 \\ E_2: & a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \ldots + a_{2n}x_n & = & b_2 \\ & \ldots & & \\ E_n: & a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \ldots + a_{nn}x_n & = & b_n \end{array} \tag{2.1}$$

que lo representaremos matricialmente de la forma

$$Ax = b \tag{2.2}$$

donde A es una matriz cuadrada de $n \times n$ que contiene los coeficientes del sistema, x es un vector columna de $n \times 1$ que contiene las incógnitas $x_1, x2, \ldots, x_n$, y b es un vector de las mismas dimensiones que x que contiene los términos independientes de las ecuaciones.

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}, \qquad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \qquad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

El objetivo de este capítulo es el estudio de diferentes métodos para la resolución de sistemas de ecuaciones lineales, analizando la estructura de la matriz de coeficientes, y describir el funcionamiento particular de cada método.

2.1. Algunos conceptos de álgebra lineal

En esta sección se introducirán algunos conceptos básicos del álgebra lineal, y algunas notaciones generales que se utilizarán de aquí en adelante.

Como vimos en el final de la introducción, denotamos con mayúsculas a las matrices (por ejemplo A) y con minúsculas a los vectores (como x y b). Las minúsculas las emplearemos para otro tipo de elementos, pero será debidamente aclarado en caso de ser necesario.

Para denotar las entradas de una matriz, se utilizará la respectiva minúscula con doble subíndice para identificar la entrada. Así, dada la matriz A, a_{ij} representa a la entrada ubicada en la i-ésima fila y j-ésima columna.

En general, una matriz A de n filas y m columnas diremos que es de $n \times m$ o bien, $A \in \mathbb{R}^{n\times m}$. Así mismo, para un vector x de n componentes (ya sea fila o columna) diremos que es n-dimensional o $x \in \mathbb{R}^n$.

Dada una matriz A (vector x), su **transpuesta** se denotará como A^T (x^T).

En la siguiente definición, se enumeran diferentes denominaciones según la estructura de las matrices, que serán utilizadas a lo largo de estas notas.

Definición 2.1. Las siguientes son definiciones referidas a matrices:

(i) Una **matriz cuadrada** es una matriz con igual número de filas y de columnas.

(ii) Una **matriz diagonal** D es una matriz cuadrada tal que sus entradas $d_{ij} = 0$ siempre que $i \neq j$.

(iii) La **matriz identidad**, que denotaremos como I, es una matriz diagonal con entradas igual a uno en su diagonal principal.

(iv) Una matriz cuadrada U se denomina **triangular superior** si sus entradas $u_{ij} = 0$ para todo $i > j$. Dicho de otra manera, las entradas por estrictamente por debajo de la diagonal son nulas.

(v) Una matriz cuadrada L se denomina **triangular inferior** si sus entradas $l_{ij} = 0$ para todo $i < j$. Dicho de otra manera, las entradas por estrictamente por arriba de la diagonal son nulas.

(vi) Una matriz cuadrada M de $n \times n$ se denomina **matriz banda** si existen dos enteros no negativos k_1 y k_2 tales que las entradas $m_{ij} = 0$ para $i - j > k_1$ ó $j - i > k_2$. En otras palabras, las entradas fuera de una *banda* alrededor de la diagonal son nulas.

(vii) Una **matriz rala** o *matriz sparse* es una matriz que tiene muchos ceros, y pocas entradas no nulas.

La notación que se utilizará para el **determinante** de una matriz cuadrada A es $\det(A)$.

Si pensamos en resolver un sistema lineal de la forma (2.2), primero debemos preguntarnos si éste tiene solución. Repasemos algunos conceptos al respecto:

Definición 2.2. Diremos que una matriz $n \times n$ A es **no singular** (o *invertible*) si existe otra matriz A^{-1} que cumple $AA^{-1} = A^{-1}A = I$. La matriz A^{-1} se denomina **inversa** de la matriz A. La matriz que no posee inversa se denominan **singular** (o *no invertible*).

El siguiente teorema nos ofrece formas alternativas para determinar si una matriz es no singular.

Teorema 2.1. *Las siguientes afirmaciones sobre una matriz A cuadrada de $n \times n$ son equivalentes:*

i. *La matriz A es no singular.*

ii. *El sistema $Ax = b$ tiene solución única para cada vector columna n-dimensional b.*

iii. *El sistema $Ax = \mathbf{0}$ tiene como única solución al vector nulo ($x = \mathbf{0}$).*

iv. $\det(A) \neq 0$.

En cuanto a los valores numéricos que podemos encontrar en las matrices cuadradas, existen tipos de matrices muy importantes de clasificar. Esto nos ayudará más adelante cuando decidamos qué métodos podemos utilizar para resolver el sistema lineal de nuestro problema.

Definición 2.3. Una matriz $n \times n$ A se dice que es **estrictamente diagonal dominante** (e.d.d.) si cumple

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^{n} |a_{ij}|, \text{ para } i = 1, 2, \ldots, n. \tag{2.3}$$

Se puede demostrar (queda como ejercicio) que si una matriz A es e.d.d., entonces es no singular.

Definición 2.4. Una matriz $A \in \mathbb{R}^{n \times n}$ simétrica es definida positiva (s.d.p.) si se cumple que:

$$x^T A x > 0 \qquad \forall x \in \mathbb{R}^n, x \neq \mathbf{0} \tag{2.4}$$

Un resultado importante para poder determinar si una matriz es s.d.p., sin tener que demostrar que se cumple la ecuación (2.4) de la definición es el siguiente teorema:

Teorema 2.2. *Una matriz A simétrica es definida positiva si, y sólo si todos sus autovalores son positivos.*

También se puede demostrar (nuevamente queda como ejercicio) que toda matriz s.d.p. es no singular.

2.1.1. Métodos de resolución

Acabamos de observar que se puede determinar si un sistema de ecuaciones lineales, representado matricialmente como $Ax = b$ tiene solución única. El objetivo ahora es poder encontrar esa solución.

Podemos clasificar a los métodos numéricos en dos grupos, que son los que estudiaremos en este curso:

- **Métodos directos**: En ellos, se realizan una cantidad finita de pasos para obtener la solución exacta del problema.
- **Métodos iterativos**: Estos construyen una sucesión de vectores que se aproxima a la solución real. Es decir, la sucesión *converge* a la solución del sistema.

En este capítulo veremos algunos métodos directos de resolución de sistemas de ecuaciones algebraicas lineales.

2.1.2. Operaciones elementales y sistemas equivalentes

Antes de estudiar los métodos directos para resolver sistemas de ecuaciones lineales veremos algunos casos particulares en los cuales podemos hallar la solución del sistema con un número reducido de operaciones.

Caso 1: Sistema con matriz diagonal.

En el caso de un sistema $Ax = b$, donde A es la matriz diagonal

$$A = \begin{bmatrix} a_{11} & 0 & 0 & \dots & 0 \\ 0 & a_{22} & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{nn} \end{bmatrix}$$

su solución es trivialmente

$$x = \begin{bmatrix} b_1/a_{11} \\ b_2/a_{22} \\ \vdots \\ b_n/a_{nn} \end{bmatrix}$$

Caso 2: Sistema con matriz triangular.

Veremos como ejemplo, el caso de una matriz triangular superior. Para el caso de una matriz triangular inferior, el procedimiento es similar.

La matriz A del sistema $Ax = b$, entonces, tiene la forma

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ 0 & a_{22} & a_{23} & \dots & a_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{nn} \end{bmatrix}$$

Podemos ver que de la última ecuación podemos obtener el valor de la componente x_n del vector incógnita x de manera directa como $x_n = b_n/a_{nn}$. Con este valor conocido, podemos obtener el valor de la componente x_{n-1} reemplazando en la penúltima ecuación el valor de x_n obtenido anteriormente.

Siguiendo este procedimiento *hacia atrás* con las ecuaciones, podemos desarrollar un procedimiento para obtener las diferentes componentes x_i calculando anteriormente las componentes x_j con $j > i$. Este proceso denominado **sustitución hacia atrás** se puede escribir formalmente como sigue:

$$\begin{aligned} x_n &= b_n/a_{nn}, \\ x_{n-1} &= (b_{n-1} - a_{n-1,n}x_n)/a_{n-1,n-1}, \\ &\vdots \\ x_i &= (b_i - \sum_{j=i+1}^{n} a_{ij}x_j)/a_{ii} \qquad \text{para } i = n-1, \ldots, 1 \end{aligned} \tag{2.5}$$

De manera similar, se resuelven sistemas con matriz triangular inferior, pero partiendo de la primer ecuación para obtener x_1 y realizando una **sustitución hacia adelante**.

En el caso de sistemas con matrices más generales, ¿se podrá utilizar algún procedimiento similar?

Antes de responder a este interrogante, introducimos el siguiente concepto. Si dos sistemas $Ax = b$ y $Bx = d$ poseen la misma solución x, se denominan **sistemas equivalentes**. Entonces, podemos reformular la pregunta anterior en la siguiente: Dado un sistema $Ax = b$ que representa a 2.1, se podrá obtener un sistema equivalente más fácil de resolver?

La respuesta es positiva, y veremos que realizando una serie de *operaciones elementales* sobre el sistema original, podemos obtener uno nuevo equivalente.

Estas *operaciones elementales* se pueden clasificar en 3 tipos:

(i) Intercambiar el orden de dos ecuaciones en el sistema: $E_i \leftrightarrow E_j$.

(ii) Multiplicar una de las ecuaciones por un escalar no nulo: $\lambda E_i \to E_i$.

(iii) Sumar a una ecuación un múltiplo de otra: $E_i + \lambda E_j \to E_i$.

El siguiente teorema nos abre camino a responder la pregunta:

Teorema 2.3. *Si un sistema de ecuaciones se obtiene a partir de otro mediante un número finito de operaciones elementales, ambos sistemas son equivalentes.*

Demostración. Es suficiente considerar el efecto que se produce al aplicar cada operación elemental. Supongamos que a nuestro sistema original $Ax = b$ se le aplica la operación (i). El sistema $Bx = d$ que se obtiene es el mismo que el original, sólo que en otro orden de ecuaciones. Por lo tanto, si x es la solución del primer sistema, también lo será del segundo, y viceversa.

Consideremos ahora la operación de tipo (ii). Entonces, supongamos que la i-ésima ecuación es la que se modifica. En este caso, esta ecuación en el sistema $Ax = b$ es

$$a_{i1}x_1 + a_{i2}x_2 + \ldots + a_{in}x_n = b_i \tag{2.6}$$

mientras que la en el sistema $Bx = d$ es

$$\lambda a_{i1}x_1 + \lambda a_{i2}x_2 + \ldots + \lambda a_{in}x_n = \lambda b_i \tag{2.7}$$

Si x es solución del primer sistema, satisface todas sus ecuaciones. En particular satisface (2.6), y por tanto, es evidente que satisface (2.7) y el resto de las ecuaciones del segundo sistema, que son comunes en ambos. De manera equivalente, si x es solución del segundo sistema, también lo será del primero.

Por último, si se realiza una operación elemental de tipo (iii). Entonces, si la j-ésima ecuación del sistema $Ax = b$ es

$$a_{j1}x_1 + a_{j2}x_2 + \ldots + a_{jn}x_n = b_j \tag{2.8}$$

la i-ésima ecuación del sistema $Bx = d$ es

$$(a_{i1} + \lambda a_{j1})x_1 + (a_{i2} + \lambda a_{j2})x_2 + \ldots + (a_{in} + \lambda a_{jn})x_n = b_i + \lambda b_j \tag{2.9}$$

y el resto de las ecuaciones son iguales en ambos sistemas.

Entonces, si x es solución de $Ax = b$, satisface (2.6) y (2.8), y por lo tanto, por lo visto en el caso anterior, podemos concluir que satisface (2.9). Luego, x es solución de $Bx = d$. A la inversa, si x es solución de $Bx = d$, satisface (2.9), y si le restamos λ veces la ecuación (2.8), se obtiene (2.6), por lo tanto x es solución del sistema $Ax = b$. ■

2.2. El método de Eliminación de Gauss

Describiremos a continuación un método muy conocido que transforma el sistema original $Ax = b$ mediante un número finito de operaciones elementales en otro sistema equivalente, cuya forma es triangular superior (y por lo tanto, fácil de resolver).

Si el sistema tiene n ecuaciones, el método de *Eliminación de Gauss* realizará $n - 1$ pasos hasta obtener un sistema con matriz triangular superior. En cada paso, se obtiene un nuevo sistema equivalente mediante una serie de operaciones elementales. Tanto la matriz A del sistema, como el vector del lado derecho b se van transformando, de tal manera que en el paso k-ésimo obtenemos un sistema equivalente $A^{(k)}x = b^{(k)}$. A continuación mostramos un ejemplo para refrescar el procedimiento, para luego desarrollar el algoritmo que implementa este método.

Ejemplo 1. Sea el sistema

$$\begin{array}{rrcrcrcrcr}
E_1: & 6x_1 & - & 2x_2 & + & 2x_3 & + & 4x_4 & = & 12 \\
E_2: & 12x_1 & - & 8x_2 & + & 6x_3 & + & 10x_4 & = & 34 \\
E_3: & 3x_1 & - & 13x_2 & + & 9x_3 & + & 3x_4 & = & 27 \\
E_4: & -6x_1 & + & 4x_2 & + & x_3 & - & 18x_4 & = & -38
\end{array}$$

que lo escribimos de forma matricial como sigue:

$$\begin{array}{c} E_1 \\ E_2 \\ E_3 \\ E_4 \end{array} \begin{bmatrix} 6 & -2 & 2 & 4 \\ 12 & -8 & 6 & 10 \\ 3 & -13 & 9 & 3 \\ -6 & 4 & 1 & -18 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 12 \\ 34 \\ 27 \\ -38 \end{bmatrix}$$

Se transformará el sistema en varios pasos realizando operaciones elementales. Como el objetivo es obtener un sistema con la matriz triangular superior, *llenaremos* de ceros debajo de la diagonal principal. Así, en el primer paso, le realizamos a cada renglón (salvo el primero) una operación de tipo (iii). Más específicamente, les restamos un múltiplo de la primer ecuación a cada una, como se muestra a continuación:

$$\begin{array}{l} E_1 \\ E_2 - 2E_1 \to E_2 \\ E_3 - \frac{1}{2}E_1 \to E_3 \\ E_4 - (-1)E_1 \to E_4 \end{array} \quad \begin{bmatrix} 6 & -2 & 2 & 4 \\ 0 & -4 & 2 & 2 \\ 0 & -12 & 8 & 1 \\ 0 & 2 & 3 & -14 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 12 \\ 10 \\ 21 \\ -26 \end{bmatrix}$$

Este primer sistema equivalente tiene una nueva matriz $A^{(1)}$ y un nuevo vector del lado derecho $b^{(1)}$. Hemos reescrito las ecuaciones $E_2, E_3 y E_4$, que por simplicidad, las seguimos nombrando de ese modo, pero se aclara que no son las mismas del sistema original.

En el segundo paso, modificamos las ecuaciones E_3 y E_4 a partir del nuevo sistema, de manera similar a lo realizado en el paso anterior, restando un múltiplo de la ecuación E_2 como se muestra a continuación:

$$\begin{array}{l} E_1 \\ E_2 \\ E_3 - 3E_2 \to E_3 \\ E_4 - (-\frac{1}{2})E_2 \to E_4 \end{array} \quad \begin{bmatrix} 6 & -2 & 2 & 4 \\ 0 & -4 & 2 & 2 \\ 0 & 0 & 2 & -5 \\ 0 & 0 & 4 & -13 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 12 \\ 10 \\ -9 \\ -21 \end{bmatrix}$$

Este nuevo sistema $A^{(2)}x = b^{(2)}$ es equivalente al anterior, y por lo tanto, al sistema original. Nuevamente, hemos reescrito las ecuaciones E_3 y E_4 (diferentes a los sistemas anteriores).

Por último, de manera similar realizamos la última operación sobre la ecuación E_4:

$$\begin{array}{l} E_1 \\ E_2 \\ E_3 \\ E_4 - 2E_3 \to E_4 \end{array} \quad \begin{bmatrix} 6 & -2 & 2 & 4 \\ 0 & -4 & 2 & 2 \\ 0 & 0 & 2 & -5 \\ 0 & 0 & 0 & -3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 12 \\ 10 \\ -9 \\ -3 \end{bmatrix}$$

A partir de este último sistema equivalente podemos obtener la solución mediante una sustitución hacia atrás como se describe en (2.5).

Lo que hemos realizado aquí es un proceso que se puede describir de la siguiente manera:

1. Se construyó una sucesión de matrices $A^{(k)}$ y vectores $b^{(k)}$ a través de operaciones elementales, construyendo sistemas equivalentes al original.

2. En el paso k, la k-ésima ecuación (o fila de la matriz) y las anteriores a ella quedan inalteradas. Sólo se modifican las posteriores.

3. En el paso $k+1$, los elementos de la matriz $A^{(k+1)}$ se calculan mediante:

$$a_{ij}^{(k+1)} = \begin{cases} a_{ij}^{(k)} & \text{para } i \leq k \\ a_{ij}^{(k)} - \left(\frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}\right) a_{kj}^{(k)} & \text{para } i \geq k+1 \text{ y } j \geq k+1 \\ 0 & \text{para } i \geq k+1 \text{ y } j \leq k \end{cases}$$

y el vector $b^{(k+1)}$ se modifica de la siguiente manera:

$$b_i^{(k+1)} = \begin{cases} b_i^{(k)} & \text{para } i \leq k \\ b_i^{(k)} - \left(\frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}\right) b_k^{(k)} & \text{para } i \geq k+1 \end{cases}$$

El elemento $a_{kk}^{(k)}$ se denomina **pivote**, la columna debajo del pivote se denomina **columna pivotal** y la fila que se utiliza para las operaciones elementales de la eliminación se denomina **fila pivotal**.

El siguiente algoritmo plantea este procedimiento, considerando una matriz aumentada $\tilde{\mathbf{A}} = [\mathbf{A}\ \mathbf{b}]$.

Entrada: n, $\tilde{\mathbf{A}} = [\mathbf{A}\ \mathbf{b}]$
Salida: $\mathbf{x}$, o mensaje de error

Eliminación

for $k = 1, 2, \ldots n-1$ do
    for $i = k+1, k+2, \ldots n$ do
        $m \leftarrow a_{ik}/a_{kk}$
        $a_{ik} \leftarrow 0$
        for $j = k+1, k+2, \ldots n+1$ do
            $a_{ij} \leftarrow a_{ij} - m\ a_{kj}$
        end
    end
end

Retrosustitución

$x_n = a_{n,n+1}/a_{nn}$
for $i = n-1, n-2, \ldots 1$ do
    $s \leftarrow a_{i,n+1}$
    for $j = i+1, i+2, \ldots n$ do
        $s \leftarrow s - a_{ij}x_j$
    end
    $x_i \leftarrow s/a_{ii}$
end

El algoritmo recorre la matriz aumentada con 3 lazos anidados: el lazo externo (variable k) recorre los pivotes, el lazo del medio (variable i) recorre las filas por debajo de la fila pivotal, y el lazo más interno (variable j) las columnas a la derecha de la columna pivotal, incluyendo la columna aumentada (modificación del vector $\mathbf{b}$).

2.2.1. Implementación en Octave

A continuación, mostramos la implementación directa en `Octave`:

```
function [x] = gauss1(A,b)
n=length(b);
A=[A b];
% Eliminacion
for k=1:n-1
  for i=k+1:n
    m = A(i,k)/A(k,k);
    A(i,k)=0;
    for j=k+1:n+1
      A(i,j) = A(i,j)-m*A(k,j);
    endfor
  endfor
endfor
if (A(n,n)==0)
  disp('no hay sol. unica')
endif
x=sust_atras1(A); %retrosustitucion
```

En la última línea de `gauss1.m` se llama a la función que resuelve el sistema triangular equivalente por el procedimiento (2.5) implementado el `Octave`:

```
function x=sust_atras1(A)
x=A(:,end); %necesario para que x sea columna
n=length(x); %definimos n por ser una variable local
x(n)=A(n,n+1)/A(n,n);
for i = n-1:-1:1
  s = A(i,n+1);
  for j = i+1:n
    s = s - A(i,j)*x(j);
  endfor
  x(i) = s/A(i,i);
endfor
```

Podemos aprovechar la capacidad que tiene `Octave` de *vectorizar* las variables y así lograr reducir los lazos anidados. La idea se centra en pensar cada paso del método de eliminación considerando las operaciones elementales como operaciones matriciales (vectoriales).

El lazo interno recorre las columnas, modificando cada elemento de la misma. Esto equivale a la operación elemental del tipo (iii). Si pensamos las ecuaciones como vectores fila, podemos reemplazar ese lazo por:

```
A(i,k+1:n+1) = A(i,k+1:n+1)-m*A(k,k+1:n+1);
```

quedando ahora 2 lazos: el interno que recorre las filas, y el externo que recorre los pivotes.

¿Podremos reemplazar un lazo más? Para responder a esta pregunta, veamos qué operaciones dependen de la variable `i` del lazo. En primer lugar se calcula el multiplicador

`m` que se usará en la operación elemental correspondiente.

Como cada operador se calcula en función del pivote, podemos crear un vector de coeficientes, cuyas entradas sean los coeficientes de cada fila debajo de la fila pivotal. Este vector se construye con el vector columna `A(k+1:n,k)` que tiene las entradas de la columna pivotal (debajo del pivote) dividido la entrada del pivote, es decir: `m=A(k+1:n,k)/A(k,k)`.

Con este vector columna, se puede modificar toda la submatriz que queda por debajo de la fila pivotal y a la derecha de la columna pivotal en una sola operación matricial: restar a esa submatriz, la matriz que se obtiene de multiplicar el vector columna `m` por el vector fila `A(k,k+1:n+1)` (que corresponde a las entradas de la fila pivotal a la derecha del pivote). Es decir, estamos realizando las operaciones elementales de tipo (iii) del paso k-ésimo del algoritmo en una sola operación matricial para todas las filas al mismo tiempo. Esto se expresa como:

```
m=A(k+1:n,k)/A(k,k);
A(k+1:n,k+1:n+1) = A(k+1:n,k+1:n+1)-m*A(k,k+1:n+1);
```

Finalmente, el proceso de eliminación queda:

```
function [x] = gauss(A,b)
n=length(b);
A=[A b];
% Eliminacion
for k=1:n-1
    m = A(k+1:n,k)/A(k,k);
    A(k+1:n,k)=0;
    A(k+1:n,k+1:n+1) = A(k+1:n,k+1:n+1)-m*A(k,k+1:n+1);
endfor
if (A(n,n)==0)
  disp('no hay sol. unica')
endif
x=sust_atras(A); %retrosustitucion
```

Podemos hacer lo mismo con el método de sustitución hacia atrás, pensando la suma como producto de vectores:

```
function x=sust_atras(A,b)
n=length(b);
x=zeros(n,1);
x(n)=b(n)/A(n,n);
for i = n-1:-1:1
  x(i) = (b(i)-A(i,i+1:n)*x(i+1:n))/A(i,i);
endfor
```

2.3. Factorización LU

Otra forma en que se puede abordar el problema de resolver el sistema $Ax = b$ es *factorizar* la matriz, es decir, buscar un par de matrices L y U tal que $A = LU$. La estructura de esas matrices debe ser tal que podamos simplificar nuestro problema. La idea principal es que la matriz L sea triangular inferior y U triangular superior. La razón es la siguiente: si se tiene esta factorización, el sistema se puede escribir como:

$$Ax = LUx = b$$

Si llamamos y al producto Ux, ahora nuestro sistema queda determinado como $Ly = b$. Este sistema posee una matriz triangular inferior L, lo cual es fácil de resolver mediante una **sustitución hacia adelante**. Al obtener y de esta manera, se calcula la solución del problema original directamente en el sistema $Ux = y$ por medio de una **sustitución hacia atrás**.

Es decir, la solución del sistema se obtiene mediante tres etapas:

1. Factorización de la matriz $A = LU$.
2. Resolución del sistema $Ly = b$ mediante una sustitución hacia adelante.
3. Resolución del sistema $Ux = y$ mediante una sustitución hacia atrás.

Hay diversas formas de obtener este tipo de factorizaciones, ya que no es única. Describiremos en esta sección una basada en la eliminación de Gauss. No es objeto de este texto profundizar en la obtención de este proceso, si no simplemente presentarlo como ejemplo.

Teorema 2.4. *Si se puede realizar la eliminación de Gauss en el sistema lineal $Ax = b$, sin intercambios de renglones, entonces, la matriz A se puede factorizar como producto de las matrices triangular inferior L y triangular superior U, $A = LU$, donde*

$$L = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ m_{21} & 1 & 0 & \dots & 0 \\ m_{31} & m_{32} & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ m_{n1} & m_{n2} & m_{n3} & \dots & 1 \end{bmatrix}$$

con $m_{ji} = a_{ji}^{(i-1)}/a_{ii}^{(i-1)}$

$$U = A^{(n-1)} = \begin{bmatrix} a_{11}^{(n-1)} & a_{12}^{(n-1)} & a_{13}^{(n-1)} & \dots & a_{1n}^{(n-1)} \\ 0 & a_{22}^{(n-1)} & a_{23}^{(n-1)} & \dots & a_{2n}^{(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{nn}^{(n-1)} \end{bmatrix}$$

Es decir, la matriz L posee unos en la diagonal principal, y almacena en cada columna debajo de la diagonal, los multiplicadores de las operaciones elementales de cada paso de la eliminación de Gauss. En tanto, la matriz U es la que se obtiene en el último paso de la eliminación.

Esta factorización, denominada **Factorización de Doolittle** es simplemente realizar el proceso de eliminación de Gauss (la matriz triangular superior obtenida es la matriz U), almacenando los multiplicadores que se utilizan en cada paso para modificar las filas (matriz L).

Es importante señalar que la eliminación de Gauss también modifica el vector de términos independientes b. Por lo tanto, si quisiera resolver un nuevo sistema que posea la misma matriz A, pero otro vector b, tenemos que repetir todo el procedimiento para obtener la solución. En cambio con la factorización esto no ocurre. El proceso de eliminación se realiza para obtener las matrices L y U una sola vez, sin modificar el vector b. Por lo tanto, si cambiara este vector, sólo debo realizar los pasos de sustitución hacia adelante y hacia atrás sin volver a realizar los pasos de eliminación.

El siguiente algoritmo en Octave implementa la factorización LU.

```
function [L,U] = doolittle(A)
n=length(A);
for k=1:n-1
    % almaceno los multiplicadores
    A(k+1:n,k) = A(k+1:n,k)/A(k,k);
    % realizo operaciones elementales
    A(k+1:n,k+1:n) = A(k+1:n,k+1:n)-A(k+1:n,k)*A(k,k+1:n);
endfor
L=tril(A,-1)+eye(n);
U=triu(A);
```

Algunas consideraciones a tener en cuenta:

- El almacenamiento se realiza sobre la misma matriz original. Los multiplicadores se almacenan el en triángulo inferior de A y en el resto de la matriz se va modificando con los pasos de eliminación.

- Los multiplicadores se calculan todos juntos, dividiendo la columna pivotal por el pivote.

- Las operaciones elementales en el paso k-ésimo se realizan con un producto de un vector columna (multiplicadores almacenados en la columna pivotal), por un vector fila (fila pivotal), realizando las operaciones elementales para todas las filas juntas.

- Una vez terminada la eliminación, se construyen, a partir de A (ya modificada) las matrices L y U.

Observación 9. En el algoritmo anterior, estamos considerando que el método *siempre funciona*, en el sentido de que todas las operaciones pueden realizarse sin problemas. Pero al calcular los multiplicadores, podemos encontrarnos con una dificultad: que un pivote sea nulo, o muy pequeño. Este detalle lo abordaremos en la siguiente sección.

2.4. Estrategias de pivoteo

Como mencionamos en el último comentario, si tenemos un pivote 0, tenemos una dificultad, ya que no podríamos realizar el cociente correspondiente para calcular los multiplicadores que constituyen la matriz L de la factorización. En el caso de que el pivote sea muy pequeño, también provoca un problema: el aumento del error de redondeo, como vimos en la sección 1.2.4. En el siguiente ejemplo se ilustra este punto:

Ejemplo 1. Sea el sistema:

$$\begin{bmatrix} \epsilon & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \qquad \text{con } \epsilon << 1$$

aplicando el método de Gauss

$$\begin{bmatrix} \epsilon & 1 \\ 0 & 1-\epsilon^{-1} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2-\epsilon^{-1} \end{bmatrix}$$

y la sustitución hacia atrás

$$x_2 = \frac{2-\epsilon^{-1}}{1-\epsilon^{-1}} \simeq 1$$

$$x_1 = (1-x_2)\ \epsilon^{-1} \simeq 0$$

Si calculamos determinantes, la solución correcta debería ser

$$x_1 = \frac{1}{1-\epsilon} \simeq 1$$

$$x_2 = \frac{1-2\epsilon}{1-\epsilon} \simeq 1$$

Luego, vemos que la solución numérica es exacta para x_2, pero no para x_1.

Entonces, si ese elemento pequeño o nulo ocasiona un inconveniente en el proceso, *lo cambiamos* por un elemento que no sea tan problemático. Recurriremos a las operaciones elementales para proceder en esta dirección. De esta manera, durante el proceso usaremos un *sistema equivalente* al original. La operación de tipo (i) (intercambio del orden de ecuaciones) es la adecuada para nuestro propósito.

2.4.1. Pivoteo parcial

El método propuesto consiste en seleccionar un elemento en la columna pivotal (debajo del pivote) que tenga el máximo valor absoluto; es decir, determinamos el menor $p \geq k$ tal forma tal que

$$|a_{pk}^{(k)}| = \operatorname*{máx}_{k \leq i \leq n} |a_{ik}^{(k)}|$$

y se realiza la operación elemental $E_k \leftrightarrow E_p$. Entonces, E_p pasa a ser la fila pivotal.

Ejemplo 2. Al aplicar este procedimiento en el sistema del Ejemplo 1, al intercambiar filas nos queda:

$$\begin{bmatrix} 1 & 1 \\ \epsilon & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

aplicando eliminación de Gauss:

$$\begin{bmatrix} 1 & 1 \\ 0 & 1-\epsilon \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1-2\epsilon \end{bmatrix}$$

. Al aplicar sustitución hacia atrás, obtenemos la solución que obtuvimos por determinantes.

Para sistemas más grandes, este intercambio en el pivote se realizará cada vez que sea necesario. Desde el punto de vista computacional, cada vez que se realiza un intercambio, se genera un costo de almacenamiento por toda la fila. Para evitar esto, no se realiza el cambio "físico" de las filas, si no que se mantiene un vector r que indica el orden en que se ha realizado la factorización. Es decir, este vector almacena el orden en que se recorrerán los pivotes.

En el ejemplo de esta sección, el vector comienza como $r = [1, 2]$, y al realizar el pivoteo, cambia a $r = [2, 1]$. Entonces, el intercambio se realiza en r y no en el sistema. De esta manera se reduce considerablemente el costo de almacenamiento.

El algoritmo de factorización con pivoteo parcial se muestra a continuación:

```
for i = 1,...n do
        r_i = i
end
for k = 1,...n-1 do                                          fila pivotal
        buscar p ∈ {k, k+1,...n}
                tal que |a_{r_p k}| = máx_{k≤i≤n} |a_{r_i k}|
        if a_{r_p k} = 0  →  mensaje de error
        if r_p ≠ r_k
                        z ← r_p
                        r_p ← r_k
                        r_k ← z

        for i = k+1,...n do
                s  ←  a_{r_i k}/a_{r_k k}
                a_{r_i k}  ←  s
                for j = k+1,...n do
                        a_{r_i j}  ←  a_{r_i j} - s a_{r_k j}
                end
        end
end
```

Vemos que las filas ya no se "recorren" en orden ascendente (según el lazo en i), si no que se realiza con el vector r, por eso vemos que la actualización de los elementos de A se realizan en $a_{r_i j}$ con el pivote $a_{r_k k}$, a diferencia de los algoritmos anteriores (a_{ij} con pivote a_{kk}.

2.4.2. Implementación en Octave

En el siguiente código, se ha implementado el método de eliminación de Gauss con sustitución hacia atrás utilizando la técnica de pivoteo parcial. Queda como ejercicio la implementación de la factorización de Doolitle.

```
function [x] = gauss_p(A,b)
n=length(b);
A=[A b];
r=1:n;
epsilon=1e-9;
for k=1:n-1
    % la funcion max devuelve
    % pmax: el pivote de mayor valor absoluto
    % p: posicion donde se encuentra pmax (local)
    [pmax,p] = max(abs(A(r(k:n),k)));
    if pmax<epsilon
      disp('Los posibles pivots son CERO')
      break
    endif
    p = p+k-1; %actualizamos pos. a numeracion global
    if p~=k
      r([p k])= r([k p]); %actualiza el pivote
    endif
    A(r(k+1:n),k) = A(r(k+1:n),k)/A(r(k),k);
    A(r(k+1:n),k+1:n+1) = A(r(k+1:n),k+1:n+1)-...
                          A(r(k+1:n),k)*A(r(k),k+1:n+1);
endfor
x=sust_atras(A(r,:));
```

Es importante que el alumno pueda identificar lo que realiza cada línea del código, y evaluar el por qué de cada diferencia respecto del código de eliminación de Gauss sin pivoteo.

2.4.3. Factorización y pivoteo

Si bien desde el algoritmo visto anteriormente se puede factorizar utilizando la estrategia de pivoteo parcial, a continuación se describe el pivoteo desde las operaciones matriciales, desde un punto de vista más conceptual.

Definición 2.5. Una bf matriz $n \times n$ de permutación P es una matriz que se obtiene de reordenar las filas de la matriz identidad I_n

Estas matrices de permutación poseen dos propiedades que son de especial interés:

- PA permuta las filas de A según la permutación de la matriz P.
- $P^{-1} = P^t$

Para ilustrar la primera, supongamos que $r_1, r_2, ..., r_n$ es un reordenamiento de los índices $1, 2, ...n$ correspondiente a las filas de la matriz identidad. Entonces, las entradas de la matriz de permutación P se definen como:

$$p_{ij} = \begin{cases} 1, & \text{si } j = r_i \\ 0, & \text{en otro caso} \end{cases}$$

Entonces, al premultiplicar A por esta matriz de permutación, se obtiene

$$PA = \begin{bmatrix} a_{r_1 1} & a_{r_1 2} & \dots & a_{r_1 n} \\ a_{r_2 1} & a_{r_2 2} & \dots & a_{r_2 n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{r_n 1} & a_{r_n 2} & \dots & a_{r_n n} \end{bmatrix}$$

es decir, la matriz A con las filas permutadas según el ordenamiento $r_!, r_2, ..., r_n$.

Entonces, si quiero resolver el sistema $Ax = b$, existe un reordenamiento del sistema original, y este nuevo sistema equivalente se puede resolver sin intercambio de filas. Más específicamente, dado $Ax = b$, existe una matriz de permutación P tal que el sistema

$$PAx = Pb$$

se resuelve sin intercambio de filas. Por lo tanto se puede factorizar esta nueva matriz:

$$PA = LU$$

con L triangular inferior y U triangular superior. Entonces, la matriz original del sistema se puede factorizar como:

$$A = P^{-1}LU = (P^t L)U$$

En este caso, si $P \neq I_n$ (es decir, se realizó al menos un intercambio de fila), la factorización de A es con U triangular superior, pero $P^t L$ no es triangular inferior (es la triangular U permutada).

Por lo tanto, para resolver el sistema $Ax = b$ mediante factorización, realizando intercambio de filas, se resuelve el sistema $Ly = Pb$ por sustitución hacia adelante, y $Ux = y$ por sustitución hacia atrás.

2.5. Factorización de Cholesky

Existe un tipo especial de matrices (ver definición 2.4) que aparece en una gran variedad de problemas, por ejemplo en mecánica de sólidos, termodinámica o en problemas estructurales, entre otros.

Se menciona a continuación un resultado que justifica un tipo especial de factorización para este caso particular. Su demostración escapa al objetivo de este apunte.

Teorema 2.5. *Una matriz simétrica A es definida positiva si, y sólo si, se puede realizar eliminación gaussiana sin intercambio de fila en el sistema lineal $Ax = b$, con todos los elementos pivotes positivos. Además, en este caso, los cálculos son estables respecto al crecimiento de errores de redondeo.*

Como corolario de este resultado, tenemos:

Corolario 2.6. *Una matriz simétrica A es definida positiva si, y sólo si A se puede factorizar en la forma LL^t, donde L es triangular inferior con entradas diagonales distintas de cero.*

Un caso particular de esta factorización lleva el nombre de *Factorización de Cholesky*, en la cual, los elementos de la digonal son todos positivos, pero diferentes de 1. Su algoritmo se describe a continuación:

Input: n, $\mathbf{A}$ (simétrica, definida positiva)
Output: $\mathbf{C}$ (sobreescrita en el triángulo inferior de $\mathbf{A}$)

$c_{11} \leftarrow \sqrt{a_{11}}$
for $i = 2, \ldots n$ do
    $c_{i1} \leftarrow a_{i1}/c_{11}$
end
for $i = 2, \ldots n-1$ do
    $$c_{ii} \leftarrow \left(a_{ii} - \sum_{s=1}^{i-1} c_{is}^2 \right)^{\frac{1}{2}}$$
    for $j = i+1, i+2, \ldots n$ do
        $$c_{ji} \leftarrow \left(a_{ji} - \sum_{s=1}^{i-1} c_{js} c_{is} \right) / c_{ii}$$
    end
end
$$c_{nn} \leftarrow \left(a_{nn} - \sum_{s=1}^{n-1} c_{ns}^2 \right)^{\frac{1}{2}}$$

Una de las ventajas de esta factorización, es que sólo se calcula una matriz, ya que el otro factor es su transpuesta. Queda como ejercicio para el alumno su implementación en `Octave`.